國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 02

姓名：林彥廷

學號：112370219

Lab02-Ex1.

Find maximum (Finds the maximum value in an array)

To-do: Count operations.

Code

```
int findMax(const vector<int>& arr) {
    int stepCount = 0;

    int max = arr[0];
    stepCount = stepCount + 2; // 2 steps (assignment and array access)

    stepCount++; // loop initialization: 1 step (assignment i = 1)
    for (int i = 1; i < arr.size(); i++) {
        stepCount++; // comparison: 1 step (i < arr.size())

        stepCount = stepCount + 2; // 2 steps (array access and comparison)
        if (arr[i] > max) {
            max = arr[i];
            stepCount = stepCount + 2; // 2 steps (array access and comparison)
        }

        stepCount++; // increment: 1 step (i++)
    }
    stepCount++; // finally failed comparison (loop exit)

    stepCount++; // return sum
    return max;
}
/*
Total operations:
3 (initial) + 6 * (n - 1) (loop operations) + 1 (loop exit) + 1 (return sum)
= 6n - 1
Therefore, O(n) complexity
*/
```

Lab02-Ex2.

Simple Nested Loops (Prints all unique pairs of elements)

To-Do: Count operations.

Code

```
void printPairs(const vector<int>& arr) {
    int stepCount = 0;

    stepCount++; // Loop initialization: i = 0
    for (int i = 0; i < arr.size(); i++) {
        stepCount++; // comparison: i < arr.size()

        stepCount++; // Loop initialization: j = i + 1
        for (int j = i + 1; j < arr.size(); j++) {
            stepCount++; // comparison: j < arr.size()

            stepCount = stepCount + 2; // 2 steps (accessing arr[i] and arr[j])
            cout << arr[i] << "," << arr[j] << endl;

            stepCount++; // increment: j++
        }
        stepCount++; // loop exit

        stepCount++; // increment: i++
    }
    stepCount++; // loop exit
}
/*
內層迴圈: (n^2-n)/2
Total operations:
1 (initial) + 4 * n (outside loop operations) + 4 * (n^2-n)/2 (inside loop
operations) + 1 (loop exit)
= 2n^2 + 2n + 2
Therefore, O(n^2) complexity
*/
```

Lab02-Ex3.

(a) Array copying code

(b) Array element counter

Code

(a)	(b)

```
void copyArray(const vector<int>&
source, vector<int>& dest) {
    int stepCount = 0;

    stepCount++; // loop
initialization (i = 0)
    for (int i = 0; i < source.size();
i++) {
        stepCount++; // comparison: i
< source.size()

        dest[i] = source[i]; stepCount
= stepCount + 3; // 2 steps array
access and 1 step  assignment

        stepCount++; // increment: i++
    }
    stepCount++; // loop exit
}
/*
Total operations:
1 (initial) + 5 * n (loop operations)
+ 1 (loop exit)
= 5n + 2
Therefore, O(n) complexity
*/
```

```
int countElement(const vector<int>&
arr, int target) {
    int stepCount = 0;

    int count = 0;
    stepCount++; // assignment (count
= 0)

    stepCount++; // loop
initialization (i = 0)
    for (int i = 0; i < arr.size();
i++) {
        stepCount++; // comparison: i
< arr.size()

        stepCount = stepCount + 2; //
array access (arr[i]) and comparison
(arr[i] == target)
        if (arr[i] == target) {
            count++; stepCount++; //
addititon (count++)
        }

        stepCount++; // increment: i++
    }
    stepCount++; // loop exit
    stepCount++; // return count
    return count;
}
/*
Total operations:
2 (initial) + 5 * n (loop operations)
+ 1 (loop exit) + 1 (return)
= 5n + 4
Therefore, O(n) complexity
*/
```

Discussion Section

1. Why don't we count operations like arr.size()?
   1) 無論 array 裡有幾個元素，執行 arr.size()所花費的時間都是一樣的
   2) arr.size()只是讀取一個記憶體位置，不涉及任何計算或迴圈。

2. Why do we drop constants in Big-O notation?
   1) 當 n 變得非常大時，多跑幾次運算對於時間影響微乎其微（3n+2≈3n），而 3n 與 n 都是線性成長，並且 Big(O)關注的是函數的增長趨勢而非精確步驟數，於是 3n 被簡化成 n
   2) Big(O)幫助我們評量演算法的好壞、直觀了解演算法的效率、如何優化演算法

3. Looking at the examples above:
   1) sumarray (5n+4)
   2) 不會，Big(O)關注的是增長趨勢，而這兩個函式皆為線性（O(n)）